

Foundations of Categorical Deep Learning: Priors via Colored PROPs and Backpropagation on Cartesian Reverse Tangent Categories

Isidor Seppälä Manning

Independent Researcher

Gothenburg, Sweden

`isidormanning@gmail.com`

ORCID: 0009-0007-2549-566X

August 1, 2026

Abstract

Categorical deep learning seeks an axiomatic foundation for neural architectures, yet current frameworks face structural limitations regarding more complex priors and general theory on backpropagation. In this paper, we formalize priors (inductive biases) and the notion of a model respecting a prior, using colored products and permutation categories (PROPs). We connect this formalism to the bottlenecks of monadic approaches while recovering geometric deep learning and other important priors. Secondly, we generalize functorial backpropagation beyond Euclidean spaces to arbitrary smooth manifolds by constructing a category of dependent lenses over cartesian reverse tangent categories (CRTCs), $\mathbf{DLens}^*(\mathcal{C})$.

1 Introduction

1.1 The Empirical Frontier of Black-Boxes

When models are scaled in data, parameters, and compute, their performance follows predictable power-law trends [Kaplan et al., 2020]. Yet beneath these smooth curves lies a fundamental "black-box problem": we can optimize networks to perfectly fit data, but we lack a causal, structural theory for why they generalize to unseen data [Simon et al., 2026].

As systems scale, they exhibit sharp phase transitions in their learning dynamics—such as grokking, double descent, and the sudden shift from rote memorization to structured generalization. Despite their prevalence, these emergent phase transitions remain mathematically unexplained. Machine learning cannot be strictly regarded as a science because its algorithms possess notable predictive power without providing structural explanations for those predictions [Wang et al., 2024, Barbierato and Gatti, 2024].

To ensure the safety, efficiency, and interpretability of future systems, we must develop a unified, axiomatic language for deep learning. We argue that a formal scientific theory requires moving beyond empirical observation to establish rigorous mathematical boundaries between memorization and generalization. With this paper, we aim to contribute to the ongoing development of the foundational abstractions and the mathematics for such a theory.

1.2 The Theoretical Frontier of Categorical Deep Learning

To move beyond black-box architectures, frameworks like **Geometric Deep Learning (GDL)** in Bronstein et al. [2021] was developed to seek a "geometric unification", providing a common mathematical framework to incorporate prior knowledge (such as symmetry and scale separation) into neural architectures. While GDL offered a first glimpse at an axiomatic theory of deep learning, where architectures are *derived* from inductive biases or "priors", it has its limitations. First, it remains fundamentally constrained by its reliance on group actions, making it difficult to apply to discrete, algorithmic, or highly compositional tasks that require different types of priors that aren't invertible. More importantly, it has been used more as an efficient framework to construct neural network architectures for a given task where certain symmetries are present, rather than an axiomatic theory that explains observations and makes falsifiable predictions.

Categorical Deep Learning (CDL) in Gavranović et al. [2024] resolves these limitations by elevating the abstraction, and ultimately paving the way towards a theory of architectures. Instead of relying on group actions, CDL uses monad algebra homomorphisms and derives the geometric priors by restricting to group action monads. In this setting, equivariance is not restricted to group symmetries; rather, a model respects a prior simply by being a morphism of algebras for a specific monad M —an M -algebra homomorphism. This allows GDL constraints to be derived from group action monad algebra homomorphisms, while also allowing for other (free) monads like List or BTree to derive Recurrent Neural Networks (RNNs) and Recursive Neural Networks (RCNNs), respectively. CDL then uses the Para construction and the category of parametric morphisms to "parameterize the models" and define weight tying. This shift toward Category Theory allows us to formulate the components of deep learning like priors, neural architectures, and optimizers, as mathematical structures, isolating the invariants that persist across different scales and domains.

1.3 Contributions and Directions

To pave a way forward through the problems in AI, we believe in an axiomatic approach where a scientific theory is built from *correct abstractions* and *restrictive mathematical assumptions*. By ensuring these foundational abstractions mirror *empirical observation*, we introduce categorical semantics for core components of deep learning. In line with this broader program, we formalize in this paper the notion of priors and architectural compliance using colored PROPs, and we generalize backpropagation to smooth manifolds via dependent lenses over Cartesian reverse tangent categories (CRTC).¹ Finally, we establish a formal categorical definition for model deployment and outline future directions for tackling open questions in generalization and interpretability.

2 A Language for Priors Using Colored PROPs

We begin by informally introducing the idea of priors before establishing the mathematical definition. By a *prior* we mean an inductive bias. Loosely speaking, it is some mathematical structure that a neural network should "understand" or "respect" in some sense. For example, an image recognition task could embed a rotation invariance prior, where rotating an image shouldn't effect the output of the model.

¹This work is heavily inspired by Gavranović [2024], Gavranović et al. [2024], and many questions raised in this paper are meant to offer new insights and perspective on questions raised in these works, in an ongoing conversation on the foundations of categorical deep learning.

We pose the *physical*² axiom that:

all machine learning tasks encode priors, which fundamentally determines equational, algebraic laws a model should respect.

No matter if the goal is to acquire Multilayer Perceptrons (MLPs) capable of modular arithmetic or LMMs capable of advanced reasoning and tool-usage, all of these tasks determine a *prior*, which determine algebraic equations. These equations ought to be satisfied by the model if we want it to learn the task.

The canonical validation for such general assumption is the idea of *geometric priors* in Geometric Deep Learning (GDL) [Bronstein et al., 2021] where it is pointed out how deriving universal network architectures directly from geometric priors hints at an axiomatization of deep learning. Moreover, it is also inspired by the observations that neural networks move from memorization to generalization by utilizing and "understanding patterns in data" as explicitly seen in, for example, the phenomena of grokking [Power et al., 2022] and double descent [Nakkiran et al., 2019]. In Gromov [2023], concrete evidence is established, showing that grokking a modular arithmetic task corresponds to learning feature maps whose mathematical structure is determined by the task.

We formalize this idea mathematically in the language of universal algebra and the category of colored Products and Permutations [nLab authors, 2026]. Readers familiar with the work outlined in Gavranović et al. [2024] will feel familiar with the general approach, since it is deeply related to the monadic approach therein. We discuss this later in Section 3.

Definition 2.1 (Colored Category of products and permutations). A $\mathcal{T}$ -colored category of products and permutations (*PROP*) is a strict symmetric monoidal category, consisting of:

1. **Objects:** finite tensor products of objects from a set of *colors* or *types* $\mathcal{T}$.
2. **Morphisms:** purely synthetic morphisms representing operations that takes a specific sequence of typed inputs and produces a specific sequence of typed outputs. The symmetry of the category provides canonical wire-crossing morphisms $\sigma_{A,B} : A \otimes B \xrightarrow{\cong} B \otimes A$.

This category is freely generated by a finite presentation consisting of a set of *generating operations* Σ and a set of *equational laws* E (e.g., commutativity, group equivariance, periodicity). We denote such generated colored PROP explicitly by $\mathcal{P} := \langle \Sigma | E \rangle$.

A generated colored PROP automatically reduces to a regular, uncolored PROP when the generating morphisms in Σ are built purely from a single color C . Examples of this include group actions $s_g : C \rightarrow C$ (Example 2.5) and addition $+$: $C \otimes C \rightarrow C$ (Example 2.7). However, this is not enough in general. In multi-modal deep learning, the priors and models can be more complex (Example 2.8).

Definition 2.2 (Colored PROP algebras and $\mathcal{P}$ -Alg). Let $\mathcal{P}$ be an $\mathcal{T}$ -colored PROP. An *algebra* or *model* of $\mathcal{P}$ in a symmetric monoidal category $\mathcal{C}$ is a strict symmetric monoidal functor $F : \mathcal{P} \rightarrow \mathcal{C}$. A *morphism* between two such algebras, $F, G : \mathcal{P} \rightarrow \mathcal{C}$, is a symmetric monoidal natural transformation $\alpha : F \Rightarrow G$. This consists of a family of morphisms $\alpha_X : F(X) \rightarrow G(X)$ in $\mathcal{C}$ for each object X in $\mathcal{P}$, such that for any morphism $f : A \rightarrow B$ in

²"Physical" in the sense that the statement of the axiom is more of a broad assumption about the empirical world of deep learning, rather than a mathematical, or *structural* axiom.

$\mathcal{P}$, the following diagram commutes:

$$\begin{array}{ccc} F(A) & \xrightarrow{F(f)} & F(B) \\ \alpha_A \downarrow & & \downarrow \alpha_B \\ G(A) & \xrightarrow{G(f)} & G(B) \end{array}$$

This defines the *category of algebras of $\mathcal{P}$ in $\mathcal{C}$* , denoted $\mathbf{Alg}(\mathcal{P}, \mathcal{C})$ or simply $\mathcal{P}\text{-}\mathbf{Alg}$ (See Figure 1).

$$\begin{array}{ccc} & M & \\ \mathcal{P} & \begin{array}{c} \curvearrowright \\ \Downarrow \alpha \\ \curvearrowleft \end{array} & \mathcal{C} \\ & N & \end{array} \qquad M \xrightarrow{\alpha} N$$

Figure 1. (Left) Functorial picture of categories $\mathcal{P}, \mathcal{C}$, with two functors $M, N : \mathcal{P} \rightarrow \mathcal{C}$ and a natural transformation $\alpha : M \Rightarrow N$ between them. (Right) M and N are objects in $\mathbf{Alg}(\mathcal{P}, \mathcal{C})$ and $\alpha : M \rightarrow N$ is a morphism in this category.

To reason about neural network architectures and weight tying, we now define a specific colored PROP $\langle \Sigma \mid E \rangle$ that, in addition to the generating operations in Σ , include two additional types of structural generators:

1. universal structural generators for copying and adding wires, and
2. a specific morphism f^* representing the task signature or the "syntactic model".

We then equip E with the standard equational laws of a commutative monoid and comonoid, allowing us to route and duplicate information across the synthetic wires before evaluating them in some base category³ via algebras.

Definition 2.3 (Priors). Let $\mathcal{T}$ be a finite set of colors and $\mathcal{C}$ a base category. A *prior* is a triple $(\mathcal{P}, M, N)$, consisting of algebras $M, N : \mathcal{P} \rightarrow \mathcal{C}^4$ and a $\mathcal{T}$ -colored PROP $\mathcal{P}$ presented by generators and equations $\langle \Sigma \mid E \rangle$, together with a distinguished morphism called the *syntactic model*:

$$f^* : \bigotimes_{i \in I} C_i \longrightarrow \bigotimes_{o \in O} C_o$$

where I and O are finite index sets indexing the input and output colors $C_i, C_o \in \mathcal{T}$. Furthermore, for every color $C \in \mathcal{T}$, the presentation $\langle \Sigma \mid E \rangle$ contains:

- a copying generator $\Delta_C : C \rightarrow C \otimes C$, and
- an adding generator $\nabla_C : C \otimes C \rightarrow C$,

such that for each C , the pairs (C, Δ_C) and (C, ∇_C) satisfy the standard axioms of a comonoid and a monoid respectively.

³Specifically, a Cartesian Reverse Tangent Category. See Section 4 where we further elaborate on the base category of deep learning.

⁴We will often refer to these as the *input algebra* and *output algebra*, respectively.

We can think of this specialized PROP as the *syntax* of the task, defining the available operations and equational laws they must satisfy. The input and output algebras provide the *semantics*, formally defining the realization of the prior in the base category $\mathcal{C}$. The algebras are what determine the prior. Intuitively, a prior conveys the fact that while $\mathcal{P}$ contains the rules, the pair (M, N) determines how models in $\mathcal{C}$ are constrained.

Including Δ or ∇ in a prior $\mathcal{P}$ intuitively specifies that tasks requires the ability to route information by both copying wires and adding them. The copy generator is the formal representation of *information broadcasting*. Even if not used in a law in E , it is required to describe models where a single layer data must influence multiple output parts simultaneously. For example, in architectures derived from geometric priors in GDL, the symmetry (the prior) forces a specific wiring where translation invariance requires weight sharing (copying Δ) across the grid. The add generator is the representation of *information aggregation*, essential for defining priors for models like Graph Convolutional Neural Networks (GCNs) where a node's state must be an aggregate of its neighbors' states (Example 2.8).

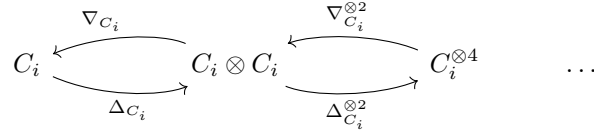


Figure 2. String diagram of the PROP generated by solely ∇ and Δ for any color C_i .

[As part of future work, I want to study the notion of instantiating an architecture "based on a prior" in some sense. Perhaps studying the notion of functors $\mathbf{Para}(M), \mathbf{Para}(N) : \mathbf{Para}(\mathcal{P}\text{-}\mathbf{Alg}) \rightarrow \mathbf{Para}(\mathcal{C})$? How we can instantiate a model in the differential structure of $\mathcal{C}$ that inherently respects a prior? During writing this I haven't really found nor explored formal examples motivating the assumption of universally including Δ and ∇ in the definition of a prior. However, I do believe it will be crucial in constructing such functorial notions of instantiation, as many tasks and architectures often require a canonical way to copy and add wires syntactically. This is to me interesting; the interpretation that tasks require the ability to route information by copying and adding maybe hints at the notion that a prior in general isn't *just* a set of equations but also a particular wiring diagram for data flow?]

Once such prior is defined, we need to understand what it means for a model to "respect the prior"? That is, how do we know a specific neural network architecture is *wired* or *composed* in such a way that it mathematically respects the equational laws in the prior? As we will see, in this context, given a prior $(\mathcal{P}, M, N)$ and a model $f : X \rightarrow Y$ in the base category $\mathcal{C}$, whether the model to "respects the prior" becomes a question of whether it can be viewed as a bridge (a natural transformation component) between M and N . More specifically, if it's the diagonal composite of a naturality square associated with its syntax f^* in $(\mathcal{P}, M, N)$. Then, if f lifts to a morphism f^* in $\mathcal{P}\text{-}\mathbf{Alg}$ via a natural transformation $\alpha : M \Rightarrow N$, then it strictly commutes with all generating operations in Σ according to the equational laws in E .

Definition 2.4 (Respecting the Prior). Let $(\mathcal{P}, M, N)$ be a prior. A model $f : X \rightarrow Y$ in $\mathcal{C}$ *respects the prior* $(\mathcal{P}, M, n)$ if there exists a monoidal natural transformation $\alpha : M \Rightarrow N$ such that f factors as the diagonal of the naturality square for its syntax f^* .

Concretely, this definition states that f respects $(\mathcal{P}, M, N)$ if there exists a morphism of

algebras $\alpha : M \rightarrow N$ in $\mathcal{P}\text{-}\mathbf{Alg}$ such that f satisfies the following equation:

$$f = N(f^*) \circ \alpha_m = \alpha_n \circ M(f^*),$$

where $m = \bigotimes_{i \in I} C_i$ and $n = \bigotimes_{o \in O} C_o$ represent the syntactic domain and codomain objects in the prior, respectively.

Empirically, f^* represents the existence of a learnable process within the task's logic. $M(f^*)$ and $N(f^*)$ are the processing steps of the task in the input and output spaces, respectively. The natural transformation α is the learned bridge where we require components that must adapt to ensure that the model f respects the prior. This formulation of ensuring a model f is the diagonal of the f^* square structurally guarantees that f is compatible with every other naturality square in the prior's $\mathcal{P}$ PROP. It shifts the prior-preserving property from the model itself to the natural transformation α , which acts as a bridge between the input and output algebra.

We should not conflate a model respecting a prior with a model generalizing on a task. They are distinct phenomena that exist on different levels of abstraction, and by using category theory we hope to eventually establish formal connections between them. Respecting a prior is a *structural property*, while *generalizing on a task* is an empirical outcome, yet we theorize they are deeply connected. More specifically, *respecting a prior* simply asserts that the internal geometry of a model aligns with the abstract syntax of a colored PROP $\mathcal{P}$, meaning its operations commute according to the equational laws. However, respecting a prior does not imply generalizing on a task. For example, a model that maps every input to a constant zero vector perfectly respects a translation invariance prior, but it has not generalized on a task like object recognition. *Generalizing*, conversely, describes the empirical outcome of a model maintaining a bounded loss when its evaluation is extended from a training dataset to unseen test data. We must rigorously separate the two. This separation stays true to the field and could allow us to formulate central questions of a predictive science of deep learning: under what conditions does forcing a network to respect a prior mathematically guarantee that it will generalize on the underlying task? What are the relations between a model generalizing on a task and a model respecting a prior? This is one part of future work we are excited to work on (Section 6).

Example 2.5 (Deriving Geometric Deep Learning). In GDL, geometric priors are derived from restricting the equational laws in E to those of a group action and restricting the colors to a single color $\mathcal{T} = \{C\}$. The signature Σ contains the syntax of a group action $s_g : C \rightarrow C$, for all $g \in G$ and a syntax $f^* : C \rightarrow C$, and E contains two laws, corresponding to the identity and composition axioms of a group action:

$$\Sigma = \left\{ \begin{array}{l} s_g : C \rightarrow C \quad \text{for all } g \in G \\ f^* : C \rightarrow C \end{array} \right\}$$

$$E = \left\{ \begin{array}{l} s_g \circ s_h = s_{gh} \quad \text{for all } g, h \in G, \\ s_e = \text{id}_C \end{array} \right\}$$

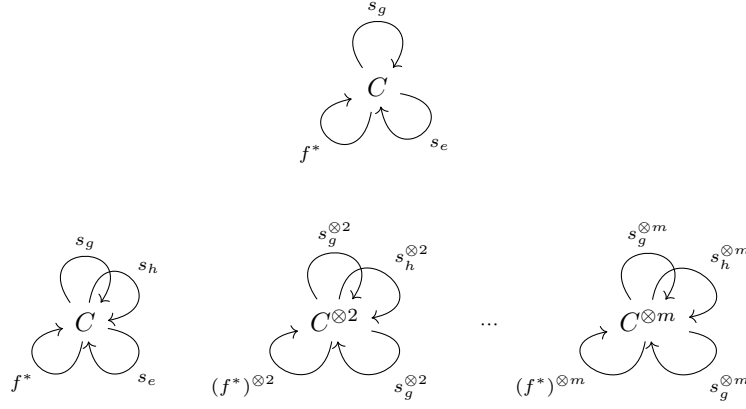


Figure 3. (Top) String diagram of the PROP generated by $\mathcal{P}_G := \langle \Sigma \mid E \rangle$. Note that not all endomorphisms are drawn. The only one drawn is for the specific group element $g \in G$. (Bottom) A more complete picture of the $\mathcal{P}_G$ including another group action s_h .

Now, consider two algebras $M, N : \mathcal{P}_G \rightarrow \mathcal{C}$, defined on objects as:

$$M(C) := X \quad \text{and} \quad N(C) := Y.$$

Then, depending on how M and N are defined, we can enforce f to be equivariant or invariant, as we only need to define $M(s_g)$ and $N(s_g)$. For equivariance, we define the algebras to map the morphism as:

$$M(s_g) := \cdot_X \quad \text{and} \quad N(s_g) := \cdot_Y,$$

or more explicitly:

$$M(s_g)(x) := g \cdot x \quad \text{and} \quad N(s_g)(y) := g \cdot y, \quad \forall g \in G.$$

$$M(f^*) = N(f^*) = \text{id}_C$$

If we want $f : X \rightarrow Y$ to respect the prior $(\mathcal{P}, M, N)$, there must exist a morphism $M \rightarrow N$, i.e., a symmetric monoidal natural transformation $\alpha : M \Rightarrow N$, which means that the following diagram must commute for all $g \in G$:

$$\begin{array}{ccc} M(C) & \xrightarrow{M(s_g)} & M(C) \\ \alpha_C \downarrow & & \downarrow \alpha_C \\ N(C) & \xrightarrow{N(s_g)} & N(C) \end{array} \quad \begin{array}{ccc} M(C) & \xrightarrow{M(f^*)} & M(C) \\ \alpha_C \downarrow & & \downarrow \alpha_C \\ N(C) & \xrightarrow{N(f^*)} & N(C) \end{array}$$

Figure 4. String diagram of PROP generated by $\mathcal{P}_G := \langle \Sigma \mid E \rangle$.

By definition, we have that:

$$\begin{aligned} f &= N(\text{id}_C) \circ \alpha_C = \alpha_C \circ M(\text{id}_C) \\ \Rightarrow f &= \alpha_C \end{aligned}$$

Thus, in this case, the model f respects the prior if it *is* the natural transformation between the algebras. This makes is to that the diagrams in Figure 4 reduces to:

$$\begin{array}{ccc} X & \xrightarrow{\cdot_X} & X \\ f \downarrow & & \downarrow f \\ Y & \xrightarrow{\cdot_Y} & Y \end{array}$$

and unpacking this diagram gives:

$$f(g \cdot_X x) = g \cdot_Y f(x).$$

One detail to note is that since a model respecting a prior is a natural transformation, we are forced to interpret it not as a single model $f : X \rightarrow Y$ but as a *set of models* $f^m : X^m \rightarrow Y^m$, one for each $m \in \mathbb{N}$. Because M and N are (symmetric) monoidal functors we get:

$$M(s_g^{\otimes m}) = M(s_g)^m = (\cdot_X)^m \quad \text{and} \quad N(s_g^{\otimes m}) = N(s_g)^m = (\cdot_Y)^m.$$

Empirically, this means that if we know that a single layer or "model unit" f^1 lifts to one *component* of a morphism $M \rightarrow N$ in $\mathcal{P}\text{-}\mathbf{Alg}$, we are guaranteed that applying the model unit to m independent spaces (in parallel) will simultaneously respect the prior. This might be useful since it results in interpretations like that a model respecting a prior generalizes across batch sizes or other contexts such as across different graph sizes.

$$\begin{array}{ccc} X^m & \xrightarrow{(\cdot_X)^m} & X^m \\ \alpha_m \downarrow & & \downarrow \alpha_n \\ Y^m & \xrightarrow{(\cdot_Y)^m} & Y^m \end{array}$$

Figure 5. If the diagram in Figure 4 commutes, then the diagram in the figure commutes as well, due to M and N being monoidal functors. Here, $\alpha_m := \alpha_1^{\otimes m}$

Invariance is not a separate structural property, and is acquired by constraining the output algebra N to be the trivial algebra:

$$N(s_g) := \text{id}_Y, \quad \text{for all } g \in G.$$

Substituting this into the diagrams in Figure 4, and then unpacking the resulting diagram, we get:

$$f(g \cdot_X x) = f(x).$$

Note that in Example 2.5, we have that:

$$M(f^*) = N(f^*) = \text{id}_C.$$

This is not always the case. The reason the model f becomes the natural transformation component α_C itself is that GDL assumes the task is nothing but the symmetry. There is no specification defined in the prior other than the group action axioms. If Σ contains only

symmetries and a placeholder f^* , then $M(f^*)$ and $N(f^*)$ will almost always be identities because there is no other "stuff" to interpret. However, if Σ contains special operators (like $+_p$ in Example 2.7) or use the generators Δ and ∇ , then the model f must factor through these concrete operations, meaning f is no longer just an equivariant map, but a map that implements some sort of *algebraic step* (like neighborhood aggregation in Example 2.8).

Note that it is also not the case that $M(f^*) = N(f^*)$ in general. In a task like Image-to-Speech, the input algebra M might interpret a "shift" as a spatial translation, while the output algebra N interprets it as a temporal shift. Here, $M(f^*)$ and $N(f^*)$ would have to be different operators to encode such types of priors.

Example 2.6 (Mixed Priors in Multi-Modal Feature Fusion Models). Consider the deployment of specialized medical diagnostic models, such as disease classification systems based on multi-modal feature fusion [Li et al., 2024]. The syntax of these systems can be complex, where multiple colors are used. For example, a model might take chest X-ray images (image modality) alongside corresponding clinical patient descriptions (text modality) to output a diagnostic classification, yielding a syntax $f^* : C_{\text{image}} \otimes C_{\text{text}} \rightarrow C_{\text{output labels}}$, perhaps implemented as a morphism $f : \mathbb{R}^{H \times W \times C} \times \mathbb{R}^{L \times D} \rightarrow \{1, 2, \dots, k\}^C$ in $\mathcal{C}$. Using colored PROPs, we can mathematically apply, for example, a rotation-invariant prior strictly to the image and a discrete translation equivariant prior to the text input.

[In the above example, we might explore more specialized priors other than generic geometric priors like rotation-invariance. Maybe there exist specific interesting priors based on domain expertise? Could we enforce such prior? How would it look like?]

Example 2.7 (Modular Arithmetic). Let our task might be modular arithmetic. That is, we want a model $f : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}_p$ that learns the mapping $(x, y) \mapsto x + y \pmod{p}$. This prompts us to ask: what are the priors embedded by this task? In Power et al. [2022], plausible priors are commutativity, associativity, and shift equivariance. The PROP contains one single color C , and we define the prior as follows:

$$\Sigma_{\text{mod}} = \left\{ \begin{array}{l} f^* : C \times C \rightarrow C, \\ s : C \rightarrow C \end{array} \right\}$$

$$E_{\text{mod}} = \left\{ \begin{array}{ll} f^* \circ \sigma_{C,C} = f^*, & \text{(commutativity)} \\ f^* \circ (f^* \otimes \text{id}_C) = + \circ (\text{id}_C \otimes f^*), & \text{(associativity)} \\ f^* \circ (s \otimes \text{id}_C) = s \circ f^* & \text{(shift equivariance)} \end{array} \right\}$$

Here, $\sigma_{C,C}$ is the component of the symmetry isomorphism from the symmetric structure of PROP, allowing us to natively switch objects.

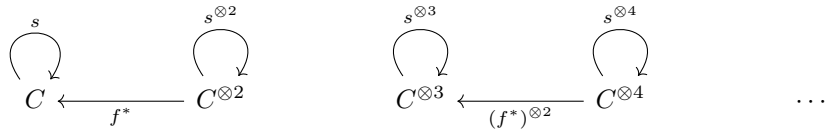


Figure 6. String diagram of the morphism $\mathcal{P}_{\text{mod}} := \langle \Sigma_{\text{mod}} \mid E_{\text{mod}} \rangle$ in a PROP.

Now, define the input and output algebras $M, N : \mathcal{P}_{\text{mod}} \rightarrow \mathcal{C}$ as follows:

$$\begin{aligned} M(1) &:= \mathbb{Z}, & M(s)(x) &:= x + 1, & M(f^*)(x, y) &:= x + y, \\ N(1) &:= \mathbb{Z}_p, & N(s)(z) &:= z + 1 \pmod{p}, & N(f^*)(z_1, z_2) &:= z_1 + z_2 \pmod{p}. \end{aligned}$$

Note that a model is of type $f : M(C \otimes C) \rightarrow N(C)$ and it respects the prior if there exists a morphism $\alpha : M \rightarrow N$ in $\mathcal{P}\text{-}\mathbf{Alg}$ such that f^* factors through the naturality squares:

$$\begin{array}{ccc}
M(C \otimes C) & \xrightarrow{M(f^*)} & M(C) \\
\alpha_{C \otimes C} \downarrow & & \downarrow \alpha_C \\
N(C \otimes C) & \xrightarrow{N(f^*)} & N(C)
\end{array}
\qquad
\begin{array}{ccc}
M(C) & \xrightarrow{M(s)} & M(C) \\
\alpha_C \downarrow & & \downarrow \alpha_C \\
N(C) & \xrightarrow{N(s)} & N(C)
\end{array}$$

$$\begin{array}{ccc}
\mathbb{Z} \times \mathbb{Z} & \xrightarrow{+z} & \mathbb{Z}_p \\
\alpha_{C \otimes C} \downarrow & & \downarrow \alpha_C \\
\mathbb{Z} \times \mathbb{Z} & \xrightarrow{+z_p} & \mathbb{Z}_p
\end{array}
\qquad
\begin{array}{ccc}
\mathbb{Z} & \xrightarrow{s_z} & \mathbb{Z} \\
\alpha_C \downarrow & & \downarrow \alpha_C \\
\mathbb{Z}_p & \xrightarrow{s_{z_p}} & \mathbb{Z}_p
\end{array}$$

Based solely on the fact that f^* must be equivariant to the shift generator s , a model respecting $\mathcal{P}_{\text{mod}}$ satisfies the following:

$$\begin{aligned}
f \circ (M(s) \otimes \text{id}_{M(C)}) &= \alpha_C \circ M(f^*) \circ (M(s) \otimes \text{id}_{M(C)}) && \text{(By definition: } f = \alpha_C \circ M(f^*)) \\
&= \alpha_C \circ M(f^* \circ (s \otimes \text{id}_C)) && \text{(Functoriality of } M) \\
&= \alpha_C \circ M(s \circ f^*) && \text{(By shift equivariance prior } E_{\text{mod}}) \\
&= \alpha_C \circ M(s) \circ M(f^*) && \text{(Functoriality of } M) \\
&= N(s) \circ \alpha_C \circ M(f^*) && \text{(By naturality of } \alpha \text{ over } s) \\
&= N(s) \circ f && \text{(Recombining back to } f)
\end{aligned}$$

This means we did not have to explicitly enforce periodicity into the network. By enforcing shift equivariance on the syntax, the actual model $f : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}_p$ is mathematically guaranteed to be invariant to shifts of p on the input if it respects this prior.

Example 2.7 gives us perhaps an angle to reason about one of the mysterious phenomena of deep learning, namely grokking. Instead of having to learn modular arithmetic by learning a massive, infinite grid of periodic invariances for every possible pair of integers (x, y) , the model's optimization has a shortcut. The network doesn't need to learn periodicity. It only needs to learn how to commute with a single shift operation s , shifting inputs by 1. The moment the model satisfies this single naturality square, we might predict that it groks on the underlying task.

[The above might be something worth exploring experimentally or looking into more research on? Could this type of result be of use in the open problem of "explaining grokking"?]

Example 2.8. For a multi-modal graph convolutional network (GCN), consider the prior $\mathcal{P}_{\text{GCN}}$ generated by:

$$\begin{aligned}
\Sigma_{\text{GCN}} &:= \left\{ \begin{array}{l} f^* : C_V \otimes C_E \rightarrow C_V, \\ s_\sigma : C_V \rightarrow C_V, \\ r_\theta : C_E \rightarrow C_E \end{array} \right\} \\
E_{\text{GCN}} &:= \left\{ \begin{array}{ll} f^* \circ (s_\sigma \otimes \text{id}_{C_E}) &= s_\sigma \circ f^*, \quad \text{(Permutation equivariance)} \\ f^* \circ (\text{id}_{C_V} \otimes r_\theta) &= f^* \quad \text{(Edge-rotation invariance)} \end{array} \right\}
\end{aligned}$$

Define:

$$\begin{aligned} M(C_V) &:= \mathbb{R}^{n \times d}, & M(C_E) &:= \mathbb{R}^{n \times n}, & M(s_\sigma) &:= P_\sigma, & M(r_\theta) &:= P_\theta \\ N(C_V) &:= \mathbb{R}^{n \times d'}, & N(C_E) &= M(C_E), & N(s_\sigma) &= M(s_\sigma), & N(r_\theta) &= \text{id}_{C_E} \end{aligned}$$

Here, P_σ and P_θ are permutation matrices.

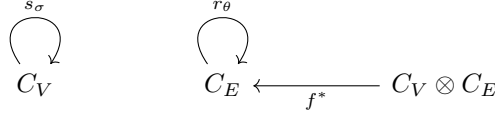


Figure 7. String diagram of the generating morphisms in $\mathcal{P}_{\text{GCN}}$.

$M(f^*)$ and $N(f^*)$ are fixed transformations in the base category. In GDL, they are usually identities. In more complex GCNs, $M(f^*)$ might represent a preprocessing step like normalizing vertex features by degree and $N(f^*)$ might be a post-processing step like applying a non-learnable projection. Because α is a monoidal natural transformation, its components at product objects are built from components at single objects (e.g., $\alpha_{C_V \otimes C_V} = \alpha_{C_V} \otimes \alpha_{C_V}$). This ties the behavior of the model f to the behavior of the individual generators in Σ_{GCN} .

In this example, the GCN must copy the node features X_i to send them across all outgoing wires to neighboring nodes j . To compute the new state for node i , the GCN must add the incoming messages from its neighborhood. This is the structural representation of information aggregation. By including Δ and ∇ in the PROP, we specify that the task requires a message-passing architecture where one piece of data influences multiple outputs and vice versa.

3 Problems with Monadic Approaches to Priors

The idea of using the general language of category theory and universal algebra to enforce priors and subsume the group-theoretic priors in GDL, is not novel. In Gavranović et al. [2024], priors are fundamentally defined using monads, and a model respects a prior if it is a monad algebra homomorphism, i.e., a morphism in the Eilenberg-Moore category of algebras and algebra homomorphisms. Using this formalism, geometric priors is the restriction to group action monad algebra homomorphisms [Gavranović et al., 2024, Example 2.6].

Although monads and PROPs they are deeply connected (See Remark 3.1), they are not equivalent. So the choice between monads and PROPs to define and work with priors comes down to a structural choice. With our goals of establishing a broader theory categorical deep learning, we argue for the approach of using PROPs instead of monads. We now elaborate on both the advantages of using PROPs and potential limitations.

One advantage with using PROPs is that it gives us a lot of flexibility. Monads $(T : \mathcal{C} \rightarrow \mathcal{C}, \mu, \eta)$ are limited to layers and models with signature $T(A) \rightarrow A$, which forces it to treat the entire space A as a single irreducible space. For instance, consider the task of modular arithmetic (See Example 2.7) where a model has the signature $f : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}_p$, which would require us to take $A = \mathbb{Z} \times \mathbb{Z}$. If we want to define a prior with equational laws like commutativity and associativity, we need to be able to access the individual input spaces $\mathbb{Z}$ of the model. PROPs can encode multi-arity equational laws natively, which do not neatly reduce to a single monad algebra. Similarly, in Example 2.8, we would have to

treat the entire graph (nodes and edges together) as a single object. This makes it difficult to define laws that act differently on specific sub-components like on vertex wires vs. edge wires. We would have to construct a monad structure map $a : T(V \times E) \rightarrow V \times E$, which results in convoluted monad algebras.

This flexibility might be used to explore a wider range priors and hence a wider range of tasks. We can think about *mixed priors* such as requiring different modalities (input spaces) of a model to obey different priors (Example 2.6) as well as more complex priors where perhaps a spatial shift in the input space must correspond to a specific rotation in the output space.

As another advantage, in group action priors and geometric deep learning (Example 2.5), we do not require the group G to exist as an object inside the base category $\mathcal{C}$. If $\mathcal{C}$ does not have a native way to represent G as a space, the group action monad used breaks as $G \times X$ is required to exist in $\mathcal{C}$. PROPs keep G entirely abstracted away from the base category $\mathcal{C}$. For example, consider a neural network $f : X \rightarrow Y$ in a base category $\mathcal{C} = \text{Poly}_{\mathbb{Z}_2}$. If we want to enforce a continuous symmetry like rotation equivariance for $G = \text{SO}(3)$, then the group G cannot exist as an object in $\mathcal{C}$, and thus $G \times X$ is not defined. Using PROPs we can define the rotation syntax abstractly (via the prior from Example 2.5 but where $G = \text{SO}(3)$) and then mapping it via input and output algebras to specific bit-flip permutations that implements the rotation logic in $\text{Poly}_{\mathbb{Z}_2}$. This would allow us to still implement "rotation priors" in $\text{Poly}_{\mathbb{Z}_2}$.

Remark 3.1 (PROPs, Lawvere theories, and Monads). It is important to establish the relations between monads and PROPs, as they are fundamentally different ways to encode algebraic theories. First, PROPs are a generalization of *Lawvere theories*.⁵ The general symmetric monoidal structure of a PROP allows it to encode multi-output operations $f : m \rightarrow n$, whereas in Lawvere theories, such operation collapses to $f : m \rightarrow 1$ by the universal property of the (cartesian) product. Since we want access to these multi-output operations (e.g., attention mechanisms), we favor PROPs over Lawvere theories. This also allows us to model complex tasks where the neural network might output many metrics (e.g. detecting objects *and* predicting their coordinates). Moreover, any Lawvere theory is equivalent to a finitary monad on **Set**.

As another example of the flexibility offered by PROPs, note that in Example 2.5, we can easily derive another type of invariance, where the group action is only applied in the codomain, i.e., where $N(s_g) := \cdot_Y$ and instead $M(s_g) := \text{id}_X$, for all $g \in G$. Diagrammatically:

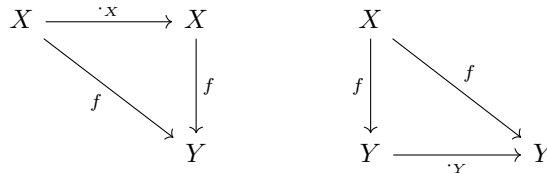


Figure 8. (Left) Invariance in the input space. (Right) Invariance in the output space.

Perhaps another advantage is that PROPs allow us to visualize our priors in string diagrams like we've seen in examples.

One limitation of PROPs is that they are structurally restricted to finitary operations (finite inputs and finite outputs), whereas monads capture both finitary and infinitary

⁵Every Lawvere theory is a PROP whose monoidal structure is Cartesian, so $(\otimes, I) = (\times, 1)$.

operations. As discussed in Gavranović et al. [2024, Example D.9], there exists infinitary monads like the covariant powerset monad whose algebras (suplattices) implements the join map for a suplattice. Such implementation requires operations for arbitrarily large cardinalities, meaning it cannot be captured by a PROP or any finitary theory in general. However, most neural network building blocks derived in Gavranović et al. [2024] such as List (RNNs), BTree (Recursive NNs), and Mealy machines rely on endofunctors and free monads that are indeed finitary. So while they acknowledge the existence of infinitary Lawvere theories for all monads, they do not utilize non-finitary monads (like the powerset monad) to derive any standard architecture. Finally, practical deep learning operates on data spaces that are inherently finite (pixels, tokens, graphs). Even though the manifolds they live on are continuous, the algebraic priors remain finitary. On a broader level, non-finitary priors are impractical as they violate the constraints of finite computation and memory.

4 Backpropagation on Cartesian Reverse Tangent Categories

We believe that a CRTC $\mathcal{C}$ [Cruttwell and Lemay, 2023, Definition 40] is a well-suited base category for deep learning, as it gives many desirable properties for modeling the empirical world of deep learning. First, $\mathcal{C}$ has finite products (i.e., is Cartesian). This is necessary for duplicating information (i.e., duplicating activations going into an attention head layer) and discarding information (i.e., discarding unused gradients). Since $\mathcal{C}$ is Cartesian, it also has a terminal object, and so it has a canonical monoidal structure $(1, \times)$. Specifically, $\mathcal{C}$ is trivially a symmetric monoidal category. By the reverse tangent structure, we get a reverse tangent bundle functor T^* that preserves products. This is necessary since if we have a parametric morphism $f : P \times A \rightarrow B$ in $\mathbf{Para}(\mathcal{C})$, T^* gives the individual types of their gradients, namely T^*A and T^*P since

$$T^*(P \times A) \cong T^*P \times T^*A.$$

Lastly, it gives us the canonical reverse differential combinator we need for the backward pass, where on $f : P \times A \rightarrow B$ it is properly defined as:

$$T^*f : A \times_B T^*B \rightarrow T^*A.$$

Remark 4.1 (The assumption of cartesianess). One caveat with CRTCs is that it makes $\mathcal{C}$ Cartesian. This means that we cannot extend the results in here to stochastic contexts, where one cannot arbitrarily copy and delete data in $\mathcal{C}$. We think this is one of the biggest limitations, and elevating the abstraction to a point where Cartesian structure isn't inherently assumed could allow us to model stochastic models and perhaps lead to applications in other fields like quantum machine learning.

One important challenge to be solved to achieve a complete categorical theory of deep learning is to construct proper categorical semantics for backpropagation. Gavranović [2024] establishes such formalism for *Euclidean spaces*, where backpropagation is derived as the coalgebra of a specific functor $\mathbf{Lens} : \mathbf{CLACat} \rightarrow \mathbf{CLACat}$ ⁶, which is a pair $(\mathcal{C}, \mathbf{R}_{\mathcal{C}})$ of both a Cartesian left-additive base category $\mathcal{C}$ and a functor $\mathbf{R}_{\mathcal{C}} : \mathcal{C} \rightarrow \mathbf{Lens}(\mathcal{C})$. This

⁶In Bruno's work, $\mathcal{C}$ is assumed to be an Additive Closed Cartesian Left-Additive Category (ACCLA), and $\mathbf{CLACat}$ is the category of cartesian left-additive categories and cartesian left-additive functors [Gavranović, 2024, Definition 6.14].

functor $\mathbf{R}_C$ yields a rigorous categorical definition for backpropagation and reverse automatic differentiation.

Machine learning is increasingly transitioning toward Geometric Deep Learning, where data resides on graphs, grids, groups, and manifolds [Bronstein et al., 2021, Papamarkou et al., 2024]. Assuming Euclideaness is thus a real restriction, as it hinders us from thinking about such setups and applying the theory to it.

To move beyond Euclidean deep learning, Gavranović [2024] points toward dependent lenses, and we will now illuminate this open problem, and connect it to reverse tangent categories. To see the structural collapses that occurs when moving from manifolds to Euclidean spaces, we break it down into three distinct simplifications:

1. In Euclidean spaces, the tangent space at any point is isomorphic to the space itself:

$$T \cong T_x X.$$

That is, the tangent Δx and the point x itself both live in $\mathbb{R}^n$, allowing us to simply add them to get a direction gradient. Moreover, this forces the reverse derivative combinator $\mathbf{R}(f)$ to resolve simply to multiplication by a transposed Jacobian matrix. However, on a manifold, tangent vectors are not the same type as points on it, so we cannot simply "add" them natively to get a gradient direction.

2. Euclidean space possesses a canonical inner product, which induces a natural isomorphism between the tangent space and the cotangent space:

$$T_x X \cong T_x^* X.$$

Therefore, gradients are treated exactly like vectors.

3. The tangent bundle of a Euclidean space is trivial:

$$T(A) \cong A \times A \quad \text{and} \quad T^*(A) \cong A \times A.$$

On a general smooth manifold, there is no canonical identification between a point and a vector, nor between a tangent vector and a cotangent vector.⁷ Thus, attempting to type a gradient update as $A \times B \rightarrow A$ on a manifold results in a type error, since we've seen now that the incoming gradient does not live in B . More specifically, it lives in the specific cotangent fiber T^*B , and instead of "adding" cotangents at points, we use pullbacks. To model this, the backward pass must be typed using a pullback structure, specifically $A \times_B T^*B \rightarrow T^*A$.

In backpropagation, we have both a forward and a backward part. However, the backward part sending gradient information backwards depends on the type of the inputs. The pullback $A \times_B T^*B$ creates a space that pairs each input point $a \in A$ with a cotangent vector specifically from the fiber over $f(a)$. The universal property of the pullback is the only way to ensure the backward pass can depend on the forward pass in a nice way. For a general smooth map $f : A \rightarrow B$, T^* produces a map of type $T^*f : A \times_B T^*B \rightarrow T^*A$. Because this requires a pullback, which is specific to the map f , we cannot compose these backward maps across the entire category $\mathcal{D}$ without referencing the forward parts. First, this means that T^* is not functorial in general. Secondly, this is why we fundamentally cannot use the same **Lens** approach. We need to study dependent lenses, where the backward morphisms *depend* on the forward ones.

⁷Unless a specific Riemannian metric is introduced.

Definition 4.2 (The Category of Dependent Lenses over a CRTC). The *category of dependent lenses* over any cartesian reverse tangent category $(\mathcal{C}, \mathbb{T}, \mathcal{D}, *, \iota)$, denoted $\mathbf{DLens}^*(\mathcal{C})$, consists of the following data:

- **Objects:** pairs $\binom{A}{p_A^*}$, where $A \in \mathcal{C}$ represents the base space (the data space), and $p_A^* : T^*A \rightarrow A$ is the canonical projection of the reverse tangent bundle defined in the system of differential bundles $\mathcal{D}$;
- **Morphisms:** a morphism from $\binom{A}{p_A^*}$ to $\binom{B}{p_B^*}$ is a pair $\binom{f}{T^*f}$, where $f \in \mathcal{C}(A, B)$ is the forward pass, and T^*f is the backward pass, evaluated as the map:

$$T^*f : A \times_B T^*B \rightarrow T^*A;$$

- **Identity:** the identity for an object $\binom{A}{p_A^*}$ is the morphism $\binom{\text{id}_A}{T^*\text{id}_A}$, where $T^*\text{id}_A = \pi_2 : A \times_A T^*A \rightarrow T^*A$ is the canonical projection.
- **Composition:** let $\binom{f}{T^*f} : \binom{A}{p_A^*} \rightarrow \binom{B}{p_B^*}$ and $\binom{g}{T^*g} : \binom{B}{p_B^*} \rightarrow \binom{C}{p_C^*}$ be two morphisms in $\mathbf{DLens}^*(\mathcal{C})$. Their composition is a morphism $\binom{h}{T^*h} : \binom{A}{p_A^*} \rightarrow \binom{C}{p_C^*}$, where $h : A \rightarrow C$ and $T^*h : A \times_C T^*C \rightarrow T^*B$ are given by:

$$\begin{aligned} h &= g \circ f, \\ T^*h &= T^*(g \circ f) = T^*f \circ \langle \pi_A, T^*g \circ \langle f \circ \pi_A, \pi_{T^*C} \rangle \rangle. \end{aligned}$$

[All proofs in this section are very detailed. They might be shortened eventually, but I thought it would be good to include as much detail as possible initially.]

Proposition 4.3 (Validity of $\mathbf{DLens}^*(\mathcal{C})$). $\mathbf{DLens}^*(\mathcal{C})$ is indeed a category of dependent lenses as defined in Gavranović [2024, Definition D.12], satisfying the associativity and unity axioms of a category.

Proof. We begin by deriving the composition law of $\mathbf{DLens}^*(\mathcal{C})$. By definition, the composition $h : A \rightarrow C$ of the forward pass is simply given by composition in the base category $\mathcal{C}$:

$$h = g \circ f$$

The composition of the backward pass requires meticulous pullback pasting because we want to route the cotangent vectors *correctly* from C back to A .

The desired backward map must have the signature $A \times_C T^*C \rightarrow T^*A$. By the universal property of the pullback, the space $A \times_C T^*C$ is equipped with projections $\pi_1 : A \times_C T^*C \rightarrow A$ and $\pi_2 : A \times_C T^*C \rightarrow T^*C$. To evaluate the first backward pass through g , we require the space $B \times_C T^*C$. We construct this by pushing the base space forward through f , pairing it with the original cotangent T^*C , which gives a map $\langle f \circ \pi_A, \pi_{T^*C} \rangle : A \times_C T^*C \rightarrow B \times_C T^*C$ by the universal property of pullbacks:

$$\begin{array}{ccccc} A \times_C T^*C & & & & \\ & \searrow \langle f \circ \pi_A, \pi_{T^*C} \rangle & & \searrow f \circ \pi_A & \\ & B \times_C T^*C & \xrightarrow{\pi_B} & B & \\ & \downarrow \pi'_{T^*C} & & \downarrow g & \\ & T^*C & \xrightarrow{p_C^*} & C & \end{array}$$

$\swarrow \pi_{T^*C}$

We then apply the backward pass of g , T^*g , generating a cotangent T^*B :

$$T^*g \circ \langle f \circ \pi_A, \pi_{T^*C} \rangle : A \times_C T^*C \rightarrow T^*B.$$

To evaluate the final backward pass through f , we require the space $A \times_B T^*B$. We now pair the original base point in A with the newly constructed cotangent T^*B , which gives a map $\langle \pi_A, T^*g \circ \langle f \circ \pi_A, \pi_{T^*C} \rangle \rangle : A \times_C T^*C \rightarrow A \times_B T^*B$ by the universal property of pullbacks:

$$\begin{array}{ccccc}
 A \times_C T^*C & \xrightarrow{\pi_A} & A & \xrightarrow{f} & B \\
 & \searrow \langle \pi_A, T^*g \circ \langle f \circ \pi_A, \pi_{T^*C} \rangle \rangle & \downarrow \pi'_A & & \\
 & & A \times_B T^*B & \xrightarrow{\pi'_A} & A \\
 & \searrow T^*g \circ \langle f \circ \pi_A, \pi_{T^*C} \rangle & \downarrow \pi'_{T^*B} & & \downarrow f \\
 & & T^*B & \xrightarrow{p_B^*} & B
 \end{array}$$

Finally, we apply the backward pass of f to generate the final cotangent vector in T^*A :

$$T^*(g \circ f) = T^*f \circ \langle \pi_A, T^*g \circ \langle f \circ \pi_A, \pi_{T^*C} \rangle \rangle.$$

To prove associativity, assume we are given a third morphism $\binom{k}{p_C^*} : \binom{C}{p_C^*} \rightarrow \binom{D}{p_D^*}$. As derived, composition in $\mathbf{DLens}^*(\mathcal{C})$ relies entirely on pullbacks and composition in the base category $\mathcal{C}$. Because the pullback pasting lemma guarantees associativity (up to isomorphism), and composition in $\mathcal{C}$ is by definition associative, the sequential evaluation of the nested backward passes T^*f , T^*g , and T^*k across the overarching pullback $A \times_D T^*D$ guarantees associativity for the morphisms.

To prove unity, let $\binom{f}{p_A^*} : \binom{A}{p_A^*} \rightarrow \binom{B}{p_B^*}$. The forward passes follow from the categorical structure of $\mathcal{C}$:

$$\text{id}_B \circ f = f = \text{id}_A \circ f.$$

The left identity is:

$$\begin{aligned}
 T^*(\text{id}_B \circ f) &= T^*f \circ \langle \pi_A, \pi_{T^*B} \circ \langle f \circ \pi_A, \pi_{T^*B} \rangle \rangle \\
 &= T^*f \circ \langle \pi_A, \pi_{T^*B} \rangle \\
 &= T^*f \circ \text{id}_{A \times_B T^*B} \\
 &= T^*f
 \end{aligned}$$

The right identity is:

$$\begin{aligned}
 T^*(f \circ \text{id}_A) &= \pi_{T^*A} \circ \langle \pi_A, T^*f \circ \langle \text{id}_A \circ \pi_A, \pi_{T^*A} \rangle \rangle \\
 &= T^*f \circ \langle \text{id}_A \circ \pi_A, \pi_{T^*B} \rangle \\
 &= T^*f \circ \langle \pi_A, \pi_{T^*B} \rangle \\
 &= T^*f \circ \text{id}_{A \times_B T^*B} \\
 &= T^*f
 \end{aligned}$$

Thus, $\mathbf{DLens}^*(\mathcal{C})$ is a valid category of dependent lenses. $\square$

[When we compose two morphisms in $\mathbf{DLens}^*(\mathcal{C})$, we compute a pullback over their shared boundary. Since pullbacks are defined by a universal property, it is only unique up to isomorphism. This means the associativity hold only up to isomorphism. Does this mean we inherently have a bicategorical structure? If we accept the notion of 1-morphisms as dependent lenses in some higher structure, what would 2-morphisms describe? Would this be useful? How does this work?]

Proposition 4.4. *The category $\mathbf{DLens}^*(\mathcal{C})$ is Cartesian.*

Proof. Since $\mathcal{C}$ is a CRTC [Cruttwell and Lemay, 2023, Definition 39], it means it has finite products (i.e., is Cartesian) [Cockett and Cruttwell, 2014, Definition 2.8], which implies the existence of a terminal object 1 in $\mathcal{C}$. By definition, T^* preserves the biproduct structure, hence:

$$T^*1 \cong 1.$$

Therefore, the terminal object in $\mathbf{DLens}^*(\mathcal{C})$ is $\left(\begin{smallmatrix} 1 \\ p_1^* \end{smallmatrix}\right)$, where the existence of $p_1^* : T^*1 \rightarrow 1$ is implied by the fact that there is an isomorphism between them. For any object $\left(\begin{smallmatrix} A \\ p_A^* \end{smallmatrix}\right)$, the unique morphism to the terminal object is $\left(\begin{smallmatrix} !_A \\ !_{T^*A} \end{smallmatrix}\right)$, where $!_A : A \rightarrow 1$ exist by definition of $\mathcal{C}$ having a terminal object, and:

$$!_{T^*A} := T^*A \xrightarrow{p_A^*} A \xrightarrow{!_A} 1 \cong T^*1$$

Thus, $\left(\begin{smallmatrix} !_A \\ !_{T^*A} \end{smallmatrix}\right)$ satisfies the universal property of terminal objects.

Let $\left(\begin{smallmatrix} A \\ p_A \end{smallmatrix}\right)$ and $\left(\begin{smallmatrix} B \\ p_B \end{smallmatrix}\right)$ be objects in $\mathbf{DLens}^*(\mathcal{C})$. Define their *product* as the object:

$$\left(\begin{smallmatrix} A \\ p_A \end{smallmatrix}\right) \times \left(\begin{smallmatrix} B \\ p_B \end{smallmatrix}\right) := \left(\begin{smallmatrix} A \times B \\ p_{A \times B} \end{smallmatrix}\right).$$

The first projection is:

$$\left(\begin{smallmatrix} \pi_A \\ T^*\pi_A \end{smallmatrix}\right) : \left(\begin{smallmatrix} A \times B \\ p_{A \times B} \end{smallmatrix}\right) \rightarrow \left(\begin{smallmatrix} A \\ p_A \end{smallmatrix}\right)$$

where the forward pass $\pi_A : A \times B \rightarrow A$ is the canonical projection from the cartesian structure of $\mathcal{C}$, and the backward pass $T^*\pi_A$ is given by:

$$T^*\pi_A = (A \times B) \times_A T^*A \xrightarrow{\langle z_B^* \circ \pi_B, \pi'_{T^*A} \rangle} T^*A \times T^*B \cong T^*(A \times B).$$

Here, the map π'_{T^*A} is provided by the universal property of the pullback, while π_B is given by the universal property of the categorical product, and $z_B^* : T^*B \rightarrow B$ is the zero map given natively by the tangent structure of $\mathcal{C}$ [Cruttwell and Lemay, 2023, Definition 1.iv]. Furthermore, we use the fact that the cotangent functor T^* distributes over products, yielding the isomorphism $T^*(A \times B) \cong T^*A \times T^*B$. The second projection $\left(\begin{smallmatrix} \pi_B \\ T^*\pi_B \end{smallmatrix}\right) : \left(\begin{smallmatrix} A \times B \\ p_{A \times B} \end{smallmatrix}\right) \rightarrow \left(\begin{smallmatrix} B \\ p_B \end{smallmatrix}\right)$ is defined similarly.

Now we need to show that the product candidate $\left(\left(\begin{smallmatrix} A \times B \\ p_{A \times B} \end{smallmatrix}\right), \left(\begin{smallmatrix} \pi_A \\ T^*\pi_A \end{smallmatrix}\right), \left(\begin{smallmatrix} \pi_B \\ T^*\pi_B \end{smallmatrix}\right)\right)$ satisfies the universal property of the product. For any other candidate

$$\left(\left(\begin{smallmatrix} C \\ p_C \end{smallmatrix}\right), \left(\begin{smallmatrix} f \\ T^*f \end{smallmatrix}\right) : \left(\begin{smallmatrix} C \\ p_C \end{smallmatrix}\right) \rightarrow \left(\begin{smallmatrix} A \\ p_A \end{smallmatrix}\right), \left(\begin{smallmatrix} g \\ T^*g \end{smallmatrix}\right) : \left(\begin{smallmatrix} C \\ p_C \end{smallmatrix}\right) \rightarrow \left(\begin{smallmatrix} B \\ p_B \end{smallmatrix}\right)\right),$$

we can define the map $\left(\begin{smallmatrix} \langle f, g \rangle \\ T^*\langle f, g \rangle \end{smallmatrix}\right)$, where the forward map $\langle f, g \rangle : C \rightarrow A \times B$ is the unique map given by $\mathcal{C}$ having finite products. For the backward pass, first note that:

$$\begin{aligned} C \times_{A \times B} T^*(A \times B) &\cong C \times_{A \times B} (T^*A \times T^*B) && \text{(distributivity of } T^*) \\ &\cong (C \times_A T^*A) \times_C (C \times_B T^*B). && \text{(preservation of limits)}^8 \end{aligned}$$

This means there exists projections

$$\begin{aligned} \pi_{C \times_A T^*A} : (C \times_A T^*A) \times_C (C \times_B T^*B) &\rightarrow C \times_A T^*A, \quad \text{and} \\ \pi_{C \times_B T^*B} : (C \times_A T^*A) \times_C (C \times_B T^*B) &\rightarrow C \times_B T^*B. \end{aligned}$$

⁸For reference, see Mac Lane [1998, Section V.4, Theorem 1].

By utilizing the given maps $T^*f : C \times_A T^*A \rightarrow T^*C$ and $T^*g : C \times_A T^*B \rightarrow T^*B$, we can compose them to get two parallel maps into T^*C :

$$\begin{aligned} T^*f \circ \pi_{C \times_A T^*A} : C \times_{A \times B} T^*(A \times B) &\rightarrow T^*C, \quad \text{and} \\ T^*g \circ \pi_{C \times_B T^*B} : C \times_{A \times B} T^*(A \times B) &\rightarrow T^*C. \end{aligned}$$

Finally, we use the sum map $s_C : T^*C \times_C T^*C \rightarrow T^*C$ from the tangent structure of $\mathcal{C}$ [Cruttwell and Lemay, 2023, definition 1.iii] which allows us to define the unique map

$$T^*\langle f, g \rangle := s_C \circ \langle T^*f \circ \pi_{C \times_A T^*A}, T^*g \circ \pi_{C \times_B T^*B} \rangle : C \times_{A \times B} T^*(A \times B) \rightarrow T^*C.$$

Diagrammatically:

$$\begin{array}{ccccc} \begin{pmatrix} A \\ p_A \end{pmatrix} & \xleftarrow{\begin{pmatrix} f \\ T^*f \end{pmatrix}} & \begin{pmatrix} C \\ p_C \end{pmatrix} & \xrightarrow{\begin{pmatrix} g \\ T^*g \end{pmatrix}} & \begin{pmatrix} B \\ p_B \end{pmatrix} \\ & \nwarrow \begin{pmatrix} \pi_A \\ T^*\pi_A \end{pmatrix} & \downarrow \begin{pmatrix} \langle f, g \rangle \\ T^*\langle f, g \rangle \end{pmatrix} & \nearrow \begin{pmatrix} \pi_B \\ T^*\pi_B \end{pmatrix} & \\ & & \begin{pmatrix} A \times B \\ p_{A \times B} \end{pmatrix} & & \end{array}$$

□

[I don't believe CRTCs are necessarily left-additive? To instantiate gradient descent on manifolds, we classically in **Smooth** need a Cartesian left-additive category whose monoid structure on objects form a group. We want to define optimizers and gradient descent in **Para(DLens^{*}(C))** in such a way that under the Euclidean trivialization, we might recover the usual notion of gradient descent in Gavranović [2024, Chapter 8.1.3].]

Proposition 4.5. *The category $\mathbf{DLens}^*(\mathcal{C})$ form a functor $\mathcal{R}_{\mathcal{C}} : \mathcal{C} \rightarrow \mathbf{DLens}^*(\mathcal{C})$, defined as follows:*

- **On objects:** since every object A in $\mathcal{C}$ comes with a canonical projection $p_A^* : T^*A \rightarrow A$, we can define:

$$\mathcal{R}_{\mathcal{C}}(A) := \begin{pmatrix} A \\ p_A^* \end{pmatrix};$$

- **On morphisms:** for any map $f : A \rightarrow B$ in $\mathcal{C}$, we define:

$$\mathcal{R}_{\mathcal{C}}(f) := \begin{pmatrix} f \\ T^*f \end{pmatrix},$$

where $T^*f : A \times_B T^*B \rightarrow T^*A$ is the reverse derivative.

Proof. We must verify that $\mathcal{R}_{\mathcal{C}}$ preserves identity morphisms and composition.

First, regarding identity preservation, the reverse tangent functor evaluated on id_A yields the canonical projection from the pullback, $T^*\text{id}_A = \pi_{T^*A} : A \times_A T^*A \rightarrow T^*A$. Because the identity morphism in $\mathbf{DLens}^*(\mathcal{C})$ is defined natively by this projection, the mapping strictly preserves identities:

$$\begin{aligned} \mathcal{R}_{\mathcal{C}}(\text{id}_A) &= \begin{pmatrix} \text{id}_A \\ T^*\text{id}_A \end{pmatrix} \\ &= \begin{pmatrix} \text{id}_A \\ \pi_{T^*A} \end{pmatrix} \\ &= \text{id}_{\begin{pmatrix} A \\ p_A^* \end{pmatrix}} \\ &= \text{id}_{\mathcal{R}_{\mathcal{C}}(A)}. \end{aligned}$$

Second, regarding compositionality, let $f : A \rightarrow B$ and $g : B \rightarrow C$ be morphisms in $\mathcal{C}$. By the axioms of a Cartesian Reverse Tangent Category, the reverse derivative of a composite map strictly satisfies the reverse chain rule:

$$T^*(g \circ f) = T^*f \circ \langle \pi_1, T^*g \circ \langle f \circ \pi_1, \pi_2 \rangle \rangle$$

In the target category $\mathbf{DLens}^*(\mathcal{C})$, the composition of morphisms is defined independently via pullback pasting. Because the sequential evaluation of pullbacks in $\mathbf{DLens}^*(\mathcal{C})$ yields this exact algebraic structure, the reverse chain rule axiom coincides with the dependent lens composition rule. Therefore, the functor maps the composition natively:

$$\begin{aligned} \mathcal{R}_{\mathcal{C}}(g \circ f) &= \left(\begin{array}{c} g \circ f \\ T^*(g \circ f) \end{array} \right) \\ &= \left(\begin{array}{c} g \circ f \\ T^*f \circ \langle \pi_A, T^*g \circ \langle f \circ \pi_A, \pi_{T^*C} \rangle \rangle \end{array} \right) \\ &= \left(\begin{array}{c} g \\ T^*g \end{array} \right) \circ \left(\begin{array}{c} f \\ T^*f \end{array} \right) \\ &= \mathcal{R}_{\mathcal{C}}(g) \circ \mathcal{R}_{\mathcal{C}}(f). \end{aligned}$$

Thus, $\mathcal{R}_{\mathcal{C}}$ strictly preserves composition and constitutes a valid functor. $\square$

[This logic of the functoriality here almost seems like a tautology. I am unsure if it's actually well-defined?]

So $\mathcal{R}_{\mathcal{C}}$ maps every object in a CRTC to its canonically associated reverse tangent bundle projection in the dual fibration, and any map $f : A \rightarrow B$ to its reverse derivative generated by the linear involution.

Note that the functor $\mathcal{R}_{\mathcal{C}} : \mathcal{C} \rightarrow \mathbf{DLens}^*(\mathcal{C})$ comes with a forgetful functor $U : \mathbf{DLens}^*(\mathcal{C}) \rightarrow \mathcal{C}$. We can easily define $U : \mathbf{DLens}^*(\mathcal{C}) \rightarrow \mathcal{C}$ as follows:

$$U \left(\begin{pmatrix} A \\ p_A^* \end{pmatrix} \right) = A, \quad U \left(\begin{pmatrix} f \\ T^*f \end{pmatrix} \right) = f.$$

That is, U is the forgetful functor that projects dependent lenses to their forward pass. This guarantees that the bidirectional and differential structure induced by $\mathbf{DLens}^*$ can be canonically forgotten via a forgetful functor to recover the original category. This will be of use to use when we later want to define the act of deploying a model categorically (See Definition 5.1).

[Does there exist a perspective in which we can view $(\mathcal{C}, \mathcal{R}_{\mathcal{C}})$ as a coalgebra of a certain of an endofunctor $\mathbf{DLens}^* : ? \rightarrow ?$ which takes any CRTC $\mathcal{C}$ and maps it to $\mathbf{DLens}^*(\mathcal{C})$. This is me trying to see if we can follow the same path as in Gavranović [2024, Theorem 6.13]. Is $\mathbf{DLens}^*(\mathcal{C})$ itself a valid CRTC? What does it all mean empirically for deep learning?]

[I want to establish connections to Bruno's coalgebras $(\mathcal{D}, \mathbf{R}_{\mathcal{D}})$ of the endofunctor $\mathbf{Lens} : \mathbf{CLACat} \rightarrow \mathbf{CLACat}$, where $\mathcal{D}$ is any cartesian left-additive category and $\mathbf{R}_{\mathcal{D}} : \mathcal{D} \rightarrow \mathbf{Lens}(\mathcal{D})$. Intuitively, if such "generalized cartesian reverse tangent category" could be established formally, I conjecture that it would "generalize" Bruno's framework in some sense, but I am not sure how this can (or cannot) be formalized?]

Proposition 4.6. *Let:*

- $\mathcal{C}$ be a CRTC,
- $\mathcal{B}$ a CRDC,

- $\mathcal{R}_{\mathcal{B}} : \mathcal{B} \rightarrow \mathbf{Lens}(\mathcal{B})$ be the functor defined in Cruttwell et al. [2021, Proposition 2.7], and
- $\mathcal{R}_{\mathcal{C}} : \mathcal{C} \rightarrow \mathbf{DLens}^*(\mathcal{C})$ be the functor defined in Proposition 4.5.

Then, there exists a structural embedding functor $\iota : \mathcal{B} \hookrightarrow \mathcal{C}$ and an induced subcategory inclusion of lenses $\mathbf{Lens}(\iota) : \mathbf{Lens}(\mathcal{B}) \hookrightarrow \mathbf{DLens}^*(\mathcal{C})$ such that the following diagram commutes:

$$\begin{array}{ccc} \mathcal{B} & \xrightarrow{\iota} & \mathcal{C} \\ \mathcal{R}_{\mathcal{B}} \downarrow & & \downarrow \mathcal{R}_{\mathcal{C}} \\ \mathbf{Lens}(\mathcal{B}) & \xrightarrow{\mathbf{Lens}(\iota)} & \mathbf{DLens}^*(\mathcal{C}) \end{array}$$

[Any help or thoughts on the attempted proof below would be appreciated. I am a bit unsure about the literature on this kind of category and result. Has it been constructed already? Has any other approaches been taken?]

Proof. By Cruttwell and Lemay [2023, Example 28], every CRDC is a trivial CRTD. For any space A and smooth map $f : A \rightarrow B$ in $\mathcal{B}$, the embedding functor ι maps them strictly to themselves:

$$\begin{aligned} \iota(A) &= A, \\ \iota(f) &= f. \end{aligned}$$

In the target category $\mathcal{C}$, this A is equipped with the trivial differential bundle. Its cotangent space is $T^*A = A \times A$, and its canonical projection is the first projection $p_A^* = \pi_1 : A \times A \rightarrow A$ ⁹. Because the pullback $A \times_B (B \times B)$ is isomorphic to $A \times B$, the backward pass T^*f trivializes to $\widetilde{T^*f} : A \times B \rightarrow A \times A$. Crucially, by Cruttwell and Lemay [2023, Proposition 42], this trivialized map is the pairing of the base projection and the CRDC reverse derivative:

$$\widetilde{T^*f} = \langle \pi_1, R[f] \rangle.$$

Applying Proposition 4.5, $\mathcal{R}_{\mathcal{C}} : \iota(\mathcal{B}) \rightarrow \mathbf{DLens}^*(\iota(\mathcal{B}))$ maps the data as:

$$\mathcal{R}_{\mathcal{C}}(A) = \begin{pmatrix} A \\ \pi_1 \end{pmatrix}, \quad \mathcal{R}_{\mathcal{C}}(f) = \begin{pmatrix} f \\ \widetilde{T^*f} \end{pmatrix}.$$

From Cockett et al. [2019, Proposition 31], $\mathcal{R}_{\mathcal{B}} : \mathcal{B} \rightarrow \mathbf{Lens}(\mathcal{B})$ maps the data as:

$$\mathcal{R}_{\mathcal{B}}(A) = \begin{pmatrix} A \\ A \end{pmatrix}, \quad \mathcal{R}_{\mathcal{B}}(f) = \begin{pmatrix} f \\ R[f] \end{pmatrix}.$$

We define the subcategory inclusion of lenses (embedding) functor $\mathbf{Lens}(\iota) : \mathbf{Lens}(\mathcal{B}) \hookrightarrow \mathbf{DLens}^*(\mathcal{C})$. On objects, it extracts the base space and typical fiber from the bundle projection. For $\pi_1 : A \times A \rightarrow A$, the fiber is A :

$$\mathbf{Lens}(\iota) \left(\begin{pmatrix} A \\ A \end{pmatrix} \right) := \begin{pmatrix} A \\ \pi_1 \end{pmatrix}.$$

On morphisms, it elevates the simple backward pass by pairing it with the base space projection:

$$\mathbf{Lens}(\iota) \left(\begin{pmatrix} f \\ R[f] \end{pmatrix} \right) := \begin{pmatrix} f \\ \langle \pi_1, R[f] \rangle \end{pmatrix} = \begin{pmatrix} f \\ \widetilde{T^*f} \end{pmatrix}.$$

⁹By convention we use the first projection since backward maps in CRDCs are defined to be "left-linear" rather than "right-linear" in the vector argument.

To prove the diagram commutes, trace an object A in $\mathcal{B}$ through the upper path:

$$A \xrightarrow{\iota} A \xrightarrow{\mathcal{R}_C} \begin{pmatrix} A \\ \pi_1 \end{pmatrix}.$$

Tracing through the lower path:

$$A \xrightarrow{\mathcal{R}_B} \begin{pmatrix} A \\ A \end{pmatrix} \xrightarrow{\mathbf{Lens}(\iota)} \begin{pmatrix} A \\ \pi_1 \end{pmatrix}.$$

The object mappings strictly commute.

On morphisms $f : A \rightarrow B$ in $\mathcal{B}$, the upper path yields:

$$f \xrightarrow{\iota} f \xrightarrow{\mathcal{R}_C} \begin{pmatrix} f \\ \widetilde{T^*f} \end{pmatrix}.$$

The lower path yields:

$$f \xrightarrow{\mathcal{R}_B} \begin{pmatrix} f \\ R[f] \end{pmatrix} \xrightarrow{\mathbf{Lens}(\iota)} \begin{pmatrix} f \\ \langle \pi_1, R[f] \rangle \end{pmatrix}.$$

Because $\widetilde{T^*f} = \langle \pi_1, R[f] \rangle$, the morphism mappings strictly commute.

Finally, we verify functoriality. For the identity map id_A , $\mathcal{R}_B(\text{id}_A) = \begin{pmatrix} \text{id}_A \\ \pi_2 \end{pmatrix}$ [Cruttwell et al., 2021, Definition 2.4], applying $\mathbf{Lens}(\iota)$ yields:

$$\mathbf{Lens}(\iota) \left(\begin{pmatrix} \text{id}_A \\ \pi_2 \end{pmatrix} \right) = \begin{pmatrix} \text{id}_A \\ \langle \pi_1, \pi_2 \rangle \end{pmatrix} = \begin{pmatrix} \text{id}_A \\ \text{id}_{A \times A} \end{pmatrix},$$

which is the exact identity morphism in $\mathbf{DLens}^*(\mathcal{C})$.

For composition of $f : A \rightarrow B$ and $g : B \rightarrow C$ in $\mathbf{Lens}(\mathcal{B})$, the reverse chain rule [Cruttwell et al., 2021, Definition 2.4] dictates:

$$\begin{pmatrix} g \\ R[g] \end{pmatrix} \circ \begin{pmatrix} f \\ R[f] \end{pmatrix} = \begin{pmatrix} g \circ f \\ R[f] \circ \langle \pi_1, R[g] \circ \langle f \circ \pi_1, \pi_2 \rangle \rangle \end{pmatrix}.$$

Applying $\mathbf{Lens}(\iota)$ to this composite yields the backward pass:

$$\langle \pi_1, R[f] \circ \langle \pi_1, R[g] \circ \langle f \circ \pi_1, \pi_2 \rangle \rangle \rangle.$$

By the universal property of products, this pairing is mathematically identical to the pullback pasting definition of composition evaluated on the trivialized bundles in $\mathbf{DLens}^*(\mathcal{C})$:

$$\widetilde{T^*f} \circ \langle \pi_1, \widetilde{T^*g} \circ \langle f \circ \pi_1, \pi_2 \rangle \rangle.$$

Thus, $\mathbf{Lens}(\iota)$ preserves composition natively, completing the proof. $\square$

This establishes a concrete connection between categorical backpropagation on Euclidean spaces and on general smooth manifolds, proving that the latter construction as given here subsumes the former one.

Note that under the trivialization ι of CRDCs into CRTCs, the subcategory in the original CRDC is left-additive (since every CRDC is left-additive).

[I want to explore the learning rate problem mentioned in Gavranović [2024]; learning rates as parametric lenses in $\mathbf{Para}(\mathbf{Lens}(\mathcal{C}))$ aren't additive in the second component, so they are not in the image of $\mathcal{R}_B : \mathcal{B} \rightarrow \mathbf{Lens}(\mathcal{B})$ where $\mathcal{B}$ is a CRDC. I haven't quite explored if $\mathcal{R}_C : \mathcal{C} \rightarrow \mathbf{DLens}^*(\mathcal{C})$ poses the same problem for implementations of learning rates.]

[I take the base category to be a cartesian one, meaning I cannot handle non-determinism. I am interested in how we might extend everything to purely a monoidal setting. Could this be tied to quantum machine learning?]

$\square$

5 Deployment and Instantiation

Since **Para** is a functorial base [Gavranović, 2024, Proposition 5.9] we can define:

$$\mathbf{Para}(\mathbf{Lens}(\mathcal{C})) \xrightarrow{\mathbf{Para}(U^*)} \mathbf{Para}(\mathcal{C}),$$

where $U : \mathbf{DLens}^*(\mathcal{C}) \rightarrow \mathcal{C}$ is the forgetful functor from the forgetful functor from $\mathbf{DLens}^*(\mathcal{C})$. In order to deploy a model, we want a functor from $\mathbf{Para}(\mathcal{C})$ to $\mathcal{C}$ by fixing a parameter state. However, we cannot define a global functor because the mapping should rely on a specific choice of weights for a specific parameter space. Instead, given a learned state morphism $p : 1 \rightarrow P^{10}$ in $\mathcal{C}$ we define an evaluation $\mathbf{Para}(\mathcal{C})(A, B) \rightarrow \mathcal{C}(A, B)$, which on parametric morphisms $f : P \times A \rightarrow B$ injects the state p and uses the left unitor isomorphism $\lambda_A : 1 \otimes A \cong A$.

Definition 5.1 (Deployment and Deployed Models). By *deploying a model* in $\mathbf{Para}(\mathbf{DLens}^*(\mathcal{C}))$ using the parameter state $p : 1 \rightarrow P$, we mean applying the functor

$$\text{Deploy}_p : \mathbf{Para}(\mathbf{DLens}^*(\mathcal{C})) \rightarrow \mathcal{C},$$

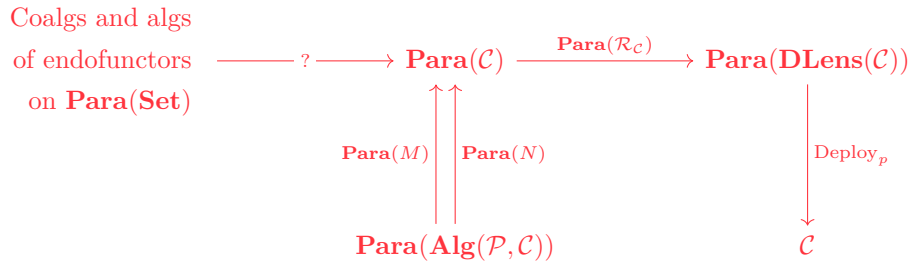
which, given a learning model $f \in \mathbf{Para}(\mathbf{DLens}^*(\mathcal{C})) \left(\left(\begin{smallmatrix} A \\ T^*A \end{smallmatrix} \right), \left(\begin{smallmatrix} B \\ T^*B \end{smallmatrix} \right) \right)$ parameterized by P , is defined by:

$$\begin{aligned} \text{Deploy}_p \left(\left(\begin{smallmatrix} A \\ T^*A \end{smallmatrix} \right) \right) &:= U^* \left(\left(\begin{smallmatrix} A \\ T^*A \end{smallmatrix} \right) \right) = A, \\ \text{Deploy}_p(f) &:= A \xrightarrow{\lambda_A^{-1}} 1 \times A \xrightarrow{p \times \text{id}_A} P \times A \xrightarrow{U^*(f)} B. \end{aligned}$$

We can technically deploy a whole training pipeline, an optimizer, a loss function, but that makes empirical no sense.

6 Reflections and Future Work

[I want to understand functorially how all of the components—priors, architectures, back-propagation, deployment—relate? See the first point of the "future work" below. Can we instantiate models from their syntax based on the (co)algebras in $\mathbf{Para}(\mathbf{Set})$ into $\mathbf{Para}(\mathcal{C})$? dependent on a defined prior? Can we then backpropagate to land in $\mathbf{Para}(\mathbf{DLens}^*(\mathcal{C}))$? I want to understand how all of these components play together, with two ultimate goals of 1) enforcing more general priors throughout the whole "pipeline of deep learning" and 2) tackling practical problems and open questions in interpretability research. Below is a very informal and ill-posed diagram. I included it to solely convey my questions and reasoning rather than any mathematical meaning:]



¹⁰This is called a *generalized element*.

Below is a list of some directions for exploration, questions, and problems we hope to work on:

- **Connecting the Components of Deep Learning:** Can we define a functorial system for deep learning, acting as a sort of "type-checker" where priors restrict the wiring of models and enforce architectural and design constraints, which would propagate to the backpropagation algorithms in $\mathbf{Para}(\mathbf{DLens}^*(\mathcal{C}))$, guaranteeing that deployed models in $\mathcal{C}$ respect the priors? We argue we need to study how these broader components (priors, instantiation, architectures, backpropagation, training, deployment, etc.) fit together in order to achieve such "verifiability system" for deep learning.
- **Software Implementations and Experiments:** We want to run more concrete experiments such as testing the modular arithmetic prior framework in code, see how enforcing priors would work in general frameworks like PyTorch, and in general establish software implementations for the theory as it is getting built and polished. We want to explore if we can implement the abstractions presented here, like reverse tangent categories and CRTCs, in programming languages like Lean (using MathLib). Having the ability to verify these abstractions formally would be extremely helpful and, on a larger scale, an important direction for AI safety and verifiable AI. As for experimentation, we are excited to research how the ideas of "mixed priors" and enforcing different priors on different parts of a model's input space (e.g. on different data modalities) could work in practical applications.
- **Categorical Deep Learning in Mechanistic Interpretability:** One of our main goals is to use the broader language of categorical deep learning to aid in open problems in interpretability research [Sharkey et al., 2025]. What type of problems in mechanistic interpretability could be studied and framed in category-theoretic terms? More importantly, for what types of problems would categorical deep learning be better equipped in constructing solutions than current methods? In what ways can CDL provide insight into problems in mechanistic interpretability in general? We are excited to explore this direction in more detail.
- **Dynamics and Optimizers:** Many phenomena like grokking, double-descent, and emergent capabilities are tied to *dynamics*. To study generalization and memorization we need to study how models change and update. The behavior of optimizers and 2-morphisms in $\mathbf{Para}(\mathbf{DLens}^*(\mathcal{C}))$ needs to be studied more rigorously in order to capture dynamics.
- **Generalization vs. Memorization:** By formalizing prior compliance through algebraic categories like that of monad algebras and/or colored PROP algebras, a next step is studying the concept of generalization categorically. What uniquely characterizes such empirical phenomena? This is connected to the note above on dynamics, since generalization inherently is a "trajectory" across parameter updates and different models. This ultimately requires us to define a notion of a "dataset" at this level of abstraction, as well as the distinction between "training datasets" and "applying a model on unseen data". It could be interesting to see how Kan extensions can help encompass these types of concepts. This also leads to interesting questions about the nature of generalization: does a model need to respect a certain prior before it can generalize on the underlying task?
- **Losses:** Since the concept of generalization is inherently tied to some notion of a "loss function", we need to study loss functions in more detail and how they live in

$\text{Para}(\text{DLens}^*(\mathcal{C}))$. Understanding the potential universal properties of loss functions is mentioned as an open question in Gavranović [2024].

- **Connections to Other Theories:** There exists many other theories of deep learning such as topological deep learning [Papamarkou et al., 2024] and neuroalgebraic geometry [Marchetti et al., 2025]. It would be interesting to explore the mathematical (axiomatic) boundaries of each field, and how category theory in deep learning might be able to provide insights into these adjacent fields. Perhaps one can ask and explore meaningful questions about deep learning that aren't fully well-posed in any single field?
- **The ReLU Activation Problem:** As in Gavranović [2024], $\mathcal{C}$ is often taken to be **Smooth**, that is the category of Euclidean spaces with morphisms corresponding to smooth maps. The problem in using this category as the base category for deep learning is that piecewise differentiable functions are not smooth. This means that they are not morphisms in **Smooth**, which in turn means that we technically cannot use activations like ReLU and other step functions. In practice, this technicality poses no problem as the derivative at cusps and corners are simply chosen arbitrarily to be some number like 0 or 1. However, if we were to implement enforce strict typing for neural networks, we need to be more careful as such arbitrarily choices are not be allowed nor mathematically valid. While using CRTCs allow us to move beyond Euclidean spaces and model networks on more spaces like manifolds, we still use smooth maps. So what is the *ideal* base category for deep learning? We would need to allow for both smooth maps but also piecewise differentiable ones, all while ensuring the category have a reverse tangent structure of some sort, allowing us to mathematically define backpropagation. In neuroalgebraic geometry, all activation functions are assumed to be polynomials, so we might turn to semi-algebraic and/or o-minimal structures. However, this also results in potentially a more and more complex (or richer) structure for the base category. We might turn to diffeological spaces, but even here, the category of diffeological spaces is not a tangent category. The subcategory of elastic diffeological spaces is a tangent category [Blohmann, 2023] but it remains an open question whether there exists a reverse tangent category on some subcategory of diffeological spaces.
- **Base Category for Deep Learning:** As expressed in Remark 4.1, it is important to be critical about the categorical structures that are empirically valid to assume. Although we want rich structures for our base category $\mathcal{C}$, we need to be ruthless about if it makes sense in the real world. For instance, in deep learning, having $\mathcal{C}$ be **Cartesian** is empirically backed up by the fact that our classical computers and machine learning models are fundamentally deterministic, so copying and deleting data or information is not a problem. However, if we were to construct a "base category for quantum machine learning", Cartesian structure would violate the No-Cloning theorem and disregard the probabilistic nature of quantum mechanics. **Closedness** would require the space of all morphisms in $\mathcal{C}$ to be objects within the category itself. This is not a problem if we want to restrict ourselves to Euclidean spaces. However, for smooth manifolds, the space of all morphisms between them is often a non-smooth, infinite-dimensional space, which breaks the assumption that $\mathcal{C}$ would be closed. I am less sure if this line of reasoning make sense, as I've seen many use cartesian closed categories as well as closed monoidal categories in applied category theory in deep learning. Maybe it isn't such a big deal? What about **completeness** and **cocompleteness**? Would these assumptions breaks down in finite compute and practical contexts? We assume finite products and pullbacks, but are there other (co)limits that are being used (or could be useful) in categorical deep learning?

References

- Enrico Barbierato and Alice Gatti. The challenges of machine learning: A critical review. *Electronics*, 13(2):416, 2024.
- Christian Blohmann. Elastic diffeological spaces, 2023. URL <https://arxiv.org/abs/2301.02583>.
- Michael M. Bronstein, Joan Bruna, Taco Cohen, and Petar Veličković. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges, 2021. URL <https://arxiv.org/abs/2104.13478>.
- J. R. B. Cockett and G. S. H. Cruttwell. Differential structure, tangent structure, and SDG. *Applied Categorical Structures*, 22(2):331–417, 2014. doi:10.1007/s10485-013-9312-0.
- Robin Cockett, Geoffrey Cruttwell, Jonathan Gallagher, Jean-Simon Pacaud Lemay, Benjamin MacAdam, Gordon Plotkin, and Dorette Pronk. Reverse derivative categories, 2019. URL <https://arxiv.org/abs/1910.07065>.
- G. S. H. Cruttwell, Bruno Gavranović, Neil Ghani, Paul Wilson, and Fabio Zanasi. Categorical foundations of gradient-based learning, 2021. URL <https://arxiv.org/abs/2103.01931>.
- Geoffrey Cruttwell and Jean-Simon Pacaud Lemay. Reverse tangent categories. 2023. URL <https://arxiv.org/abs/2308.01131>.
- Bruno Gavranović. Fundamental components of deep learning: A category-theoretic approach, 2024. URL <https://arxiv.org/abs/2403.13001>.
- Bruno Gavranović, Paul Lessard, Andrew Dudzik, Tamara von Glehn, João G. M. Araújo, and Petar Veličković. Position: Categorical deep learning is an algebraic theory of all architectures, 2024. URL <https://arxiv.org/abs/2402.15332>.
- Andrey Gromov. Grokking modular arithmetic, 2023. URL <https://arxiv.org/abs/2301.02679>.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>.
- Yihao Li, Mostafa El Habib Daho, Pierre-Henri Conze, Rachid Zeghlache, Hugo Le Boité, Ramin Tadayoni, Béatrice Cochener, Mathieu Lamard, and Gwenolé Quéllec. A review of deep learning-based information fusion techniques for multimodal medical image classification, 2024. URL <https://arxiv.org/abs/2404.15022>.
- Saunders Mac Lane. *Categories for the working mathematician*, volume 5. Springer Science & Business Media, New York, NY, 2nd edition, 1998.
- Giovanni Luca Marchetti, Vahid Shahverdi, Stefano Mereta, Matthew Trager, and Kathlén Kohn. Algebra unveils deep learning – an invitation to neuroalgebraic geometry, 2025. URL <https://arxiv.org/abs/2501.18915>.
- Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt, 2019. URL <https://arxiv.org/abs/1912.02292>.

- nLab authors. PROP. <https://ncatlab.org/nlab/show/PROP>, July 2026. Revision 30.
- Theodore Papamarkou, Tolga Birdal, Michael Bronstein, Gunnar Carlsson, Justin Curry, Yue Gao, Mustafa Hajij, Roland Kwitt, Pietro Liò, Paolo Di Lorenzo, Vasileios Maroulas, Nina Miolane, Farzana Nasrin, Karthikeyan Natesan Ramamurthy, Bastian Rieck, Simone Scardapane, Michael T. Schaub, Petar Veličković, Bei Wang, Yusu Wang, Guo-Wei Wei, and Ghada Zamzmi. Position: Topological deep learning is the new frontier for relational learning, 2024. URL <https://arxiv.org/abs/2402.08871>.
- Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets, 2022. URL <https://arxiv.org/abs/2201.02177>.
- Lee Sharkey, Bilal Chughtai, Joshua Batson, Jack Lindsey, Jeff Wu, Lucius Bushnaq, Nicholas Goldowsky-Dill, Stefan Heimersheim, Alejandro Ortega, Joseph Bloom, Stella Biderman, Adria Garriga-Alonso, Arthur Conmy, Neel Nanda, Jessica Rumbelow, Martin Wattenberg, Nandi Schoots, Joseph Miller, Eric J. Michaud, Stephen Casper, Max Tegmark, William Saunders, David Bau, Eric Todd, Atticus Geiger, Mor Geva, Jesse Hoogland, Daniel Murfet, and Tom McGrath. Open problems in mechanistic interpretability, 2025. URL <https://arxiv.org/abs/2501.16496>.
- Jamie Simon, Daniel Kunin, Alexander Atanasov, Enric Boix-Adserà, Blake Bordelon, Jeremy Cohen, Nikhil Ghosh, Florentin Guth, Arthur Jacot, Mason Kamb, Dhruva Karkada, Eric J. Michaud, Berkan Ottlik, and Joseph Turnbull. There will be a scientific theory of deep learning, 2026. URL <https://arxiv.org/abs/2604.21691>.
- Cheng Wang, Yi Wang, Zhiwei Li, Qilong Jia, and Wei Liu. Scaling laws of data-driven machine learning models: A survey and taxonomy. *Authorea Preprints*, 2024.